

Exercise Set 1

Exercise 1

- a) I chose MacOS as my operating system. As it is the one I am using on my personal computer.
- b) Running processes in MacOS can be viewed using the “Activity Monitor” application.
- c) I chose “Code” or Visual Studio Code as my process. Relevant information includes:
 - Name: Code
 - User: dmitriigorovoi
 - Process ID (PID): 54820
 - CPU usage: 4,1%
 - CPU time: 19.21
 - Memory usage: 125,8 MB
 - Number of threads: 46

Exercise 2

- a) When process is created this happens:
 - Assign an identity: create a new process ID (PID).
 - Create and initialize the process control block (PCB).
 - Set up the execution context: initialize the program counter, registers, stack pointer, and initial CPU context so it can run.
 - Provide resources and address space: create the process address space , set up memory mappings, and apply limits/permissions.
 - Inherit or attach OS objects: copy or share things like open files, current directory, environment, credentials, and IPC handles depending on the creation method.
 - Make it schedulable: set state to Ready and place it into the appropriate scheduler queue(s) so it can be dispatched.
- b)
 - Normal completion: The program finishes its work and calls an exit routine

- Error or exception: The process hits a runtime error the OS cannot safely recover from, like an illegal instruction, segmentation fault, divide-by-zero, or failed protection check.
- Killed by another entity: A user, administrator, parent process, or the OS requests termination

Exercise 3

a) To be able to resume the process exactly where it stopped, the OS must save the user-visible CPU context:

- R0, R1, R2, R3 (general registers)
- SP (stack pointer)
- FP (frame pointer)
- Return address for the user code

b) When an interrupt occurs while a user process is running, the TOY CPU automatically:

1. Copies PC -> EPC (saves where the user process should resume)
2. Loads PC with the interrupt handler address
3. Switches to system mode
4. Starts executing the interrupt handler

c)

1. Restores the saved user registers (R0-R3, SP, FP) from the PCB
2. Ensures the saved return address is in EPC
3. Executes the special “return to user” instruction, which:
 - Copies EPC -> PC
 - Switches to user mode
 - Continues execution at that PC (right after the interrupted instruction)

Exercise 4

a)

```

• (venv) dmitriigorovoi@Dmitriis-MacBook-Air HW1 % python3 process-run.py -h
Usage: process-run.py [options]

Options:
  -h, --help            show this help message and exit
  -s SEED, --seed=SEED  the random seed
  -P PROGRAM, --program=PROGRAM
                        more specific controls over programs
  -l PROCESS_LIST, --processlist=PROCESS_LIST
                        a comma-separated list of processes to run, in the
                        form X1:Y1,X2:Y2,... where X is the number of
                        instructions that process should run, and Y the
                        chances (from 0 to 100) that an instruction will use
                        the CPU or issue an IO (i.e., if Y is 100, a process
                        will ONLY use the CPU and issue no I/Os; if Y is 0, a
                        process will only issue I/Os)
  -L IO_LENGTH, --iolength=IO_LENGTH
                        how long an IO takes
  -S PROCESS_SWITCH_BEHAVIOR, --switch=PROCESS_SWITCH_BEHAVIOR
                        when to switch between processes: SWITCH_ON_IO,
                        SWITCH_ON_END
  -I IO_DONE_BEHAVIOR, --iodone=IO_DONE_BEHAVIOR
                        type of behavior when IO ends: IO_RUN_LATER,
                        IO_RUN_IMMEDIATE
  -c                    compute answers for me
  -p, --printstats      print statistics at end; only useful with -c flag
                        (otherwise stats are not printed)

```

b)

1) Expected 100% As there are no I/O operations

When ran `./process-run.py -l 5:100,5:100 -c -p`, the output was:

Time	PID: 0	PID: 1	CPU	IOs
1	RUN:cpu	READY	1	
2	RUN:cpu	READY	1	
3	RUN:cpu	READY	1	
4	RUN:cpu	READY	1	
5	RUN:cpu	READY	1	
6	DONE	RUN:cpu	1	
7	DONE	RUN:cpu	1	
8	DONE	RUN:cpu	1	
9	DONE	RUN:cpu	1	
10	DONE	RUN:cpu	1	

Stats: Total Time 10
 Stats: CPU Busy 10 (100.00%)
 Stats: IO Busy 0 (0.00%)

which matches the expectation.

- 2) Expected $4 + 1 + 5 + 1 = 11$ ticks Where 4 ticks for 1st process, 1 tick to run I/O, for 5 ticks process is blocked because of the I/O, and 1 tick to run I/O Done

When ran `./process-run.py -l 4:100,1:0 -c -p`, the output was:

Time	PID: 0	PID: 1	CPU	I/Os
1	RUN:cpu	READY	1	
2	RUN:cpu	READY	1	
3	RUN:cpu	READY	1	
4	RUN:cpu	READY	1	
5	DONE	RUN:io	1	
6	DONE	BLOCKED		1
7	DONE	BLOCKED		1
8	DONE	BLOCKED		1
9	DONE	BLOCKED		1
10	DONE	BLOCKED		1
11*	DONE	RUN:io_done	1	
Stats: Total Time 11				
Stats: CPU Busy 6 (54.55%)				
Stats: I/O Busy 5 (45.45%)				

which matches the expectation.

Exercise 5

- 3) When the order is switched processes can run in parallel, meaning that while one process is waiting for I/O the other process can use the CPU.

When ran `./process-run.py -l 1:0,4:100 -c -p`, the output was:

Time	PID: 0	PID: 1	CPU	I/Os
1	RUN:io	READY	1	
2	BLOCKED	RUN:cpu	1	1
3	BLOCKED	RUN:cpu	1	1
4	BLOCKED	RUN:cpu	1	1
5	BLOCKED	RUN:cpu	1	1
6	BLOCKED	DONE		1
7*	RUN:io_done	DONE	1	

Stats: Total Time 7
 Stats: CPU Busy 6 (85.71%)
 Stats: IO Busy 5 (71.43%)

- 4) When -S SWITCH_ON_END is used CPU won't switch to another process until I/O is done so the time will be the same as in number 2.

When ran `./process-run.py -l 1:0,4:100 -c -p -S SWITCH_ON_END`, the output was:

```

(venv) dmitriigorovoi@Dmitriis-MacBook-Air HW1 % python3 ./process-run.py -l 1:0,4:100 -c -p -S SWITCH_ON_END

```

Time	PID: 0	PID: 1	CPU	I/Os
1	RUN:io	READY	1	
2	BLOCKED	READY		1
3	BLOCKED	READY		1
4	BLOCKED	READY		1
5	BLOCKED	READY		1
6	BLOCKED	READY		1
7*	RUN:io_done	READY	1	
8	DONE	RUN:cpu	1	
9	DONE	RUN:cpu	1	
10	DONE	RUN:cpu	1	
11	DONE	RUN:cpu	1	

Stats: Total Time 11
 Stats: CPU Busy 6 (54.55%)
 Stats: IO Busy 5 (45.45%)

- 5) When -S SWITCH_ON_IO is used CPU will switch to another process as soon as I/O starts so the time will be the same as in number 3.
- 6) When -I IO_RUN_LATER is used io_done will be done after CPU processes finish which means long waiting time (when process is READY but io_done was not called yet). The resources are not effectively used as PID 0 is idle for a while.

When ran `./process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH_ON_IO -I IO_RUN_LATER -c -p`, the output was:

Time	PID: 0	PID: 1	PID: 2	PID: 3	CPU	I/Os
1	RUN:io	READY	READY	READY	1	
2	BLOCKED	RUN:cpu	READY	READY	1	1
3	BLOCKED	RUN:cpu	READY	READY	1	1
4	BLOCKED	RUN:cpu	READY	READY	1	1
5	BLOCKED	RUN:cpu	READY	READY	1	1
6	BLOCKED	RUN:cpu	READY	READY	1	1
7*	READY	DONE	RUN:cpu	READY	1	
8	READY	DONE	RUN:cpu	READY	1	
9	READY	DONE	RUN:cpu	READY	1	
10	READY	DONE	RUN:cpu	READY	1	
11	READY	DONE	RUN:cpu	READY	1	
12	READY	DONE	DONE	RUN:cpu	1	
13	READY	DONE	DONE	RUN:cpu	1	
14	READY	DONE	DONE	RUN:cpu	1	
15	READY	DONE	DONE	RUN:cpu	1	
16	READY	DONE	DONE	RUN:cpu	1	
17	RUN:io_done	DONE	DONE	DONE	1	
18	RUN:io	DONE	DONE	DONE	1	
19	BLOCKED	DONE	DONE	DONE		1
20	BLOCKED	DONE	DONE	DONE		1
21	BLOCKED	DONE	DONE	DONE		1
22	BLOCKED	DONE	DONE	DONE		1
23	BLOCKED	DONE	DONE	DONE		1
24*	RUN:io_done	DONE	DONE	DONE	1	
25	RUN:io	DONE	DONE	DONE	1	
26	BLOCKED	DONE	DONE	DONE		1
27	BLOCKED	DONE	DONE	DONE		1
28	BLOCKED	DONE	DONE	DONE		1
29	BLOCKED	DONE	DONE	DONE		1
30	BLOCKED	DONE	DONE	DONE		1
31*	RUN:io_done	DONE	DONE	DONE	1	

Stats: Total Time 31
Stats: CPU Busy 21 (67.74%)
Stats: I/O Busy 15 (48.39%)

- 7) When -I IO_RUN_IMMEDIATE is used io_done will be called as soon as I/O finishes which means next I/O can start right away after previous one finishes. The resources are used more effectively as PID 0 is idle for a shorter time. This allows to more overlap between CPU and I/O operations.

When ran `./process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH_ON_IO -I IO_RUN_IMMEDIATELY -c -p`, the output was:

```

● (venv) dmitriigorovoi@Dmitriis-MacBook-Air HW1 % python3 ./process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH_ON_IO -I IO_RUN_IMMEDIATE -c -p
Time    PID: 0      PID: 1      PID: 2      PID: 3      CPU      IOs
1       RUN:io      READY      READY      READY      1
2       BLOCKED   RUN:cpu    READY      READY      1      1
3       BLOCKED   RUN:cpu    READY      READY      1      1
4       BLOCKED   RUN:cpu    READY      READY      1      1
5       BLOCKED   RUN:cpu    READY      READY      1      1
6       BLOCKED   RUN:cpu    READY      READY      1      1
7*      RUN:io_done  DONE      READY      READY      1
8       RUN:io      DONE      READY      READY      1
9       BLOCKED   DONE      RUN:cpu    READY      1      1
10      BLOCKED   DONE      RUN:cpu    READY      1      1
11      BLOCKED   DONE      RUN:cpu    READY      1      1
12      BLOCKED   DONE      RUN:cpu    READY      1      1
13      BLOCKED   DONE      RUN:cpu    READY      1      1
14*     RUN:io_done  DONE      DONE      READY      1
15      RUN:io      DONE      DONE      READY      1
16      BLOCKED   DONE      DONE      RUN:cpu    1      1
17      BLOCKED   DONE      DONE      RUN:cpu    1      1
18      BLOCKED   DONE      DONE      RUN:cpu    1      1
19      BLOCKED   DONE      DONE      RUN:cpu    1      1
20      BLOCKED   DONE      DONE      RUN:cpu    1      1
21*     RUN:io_done  DONE      DONE      DONE      1
Stats: Total Time 21
Stats: CPU Busy 21 (100.00%)
Stats: IO Busy 15 (71.43%)

```